

1. Promise Creation

A Promise is created using the `Promise` constructor:

```
const myPromise = new Promise((resolve, reject) => {
  const success = true;
  if (success) {
    resolve("Task completed successfully!");
  } else {
    reject("Task failed.");
  }
});
```

- `resolve(value)` → Marks promise as fulfilled.
- `reject(reason)` → Marks promise as rejected.

2. .then()

Attaches a callback for fulfilled state and allows chaining.

```
myPromise
  .then(result => {
    console.log(result);
    return "Next task";
  })
  .then(next => console.log(next));
```

3. .catch()

Attaches a callback for rejected state.

```
myPromise
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

4. .finally()

Executes callback regardless of fulfillment or rejection.

```
myPromise
  .finally(() => console.log("Promise settled"))
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

5. Promise.resolve(value)

Returns a promise already resolved with `value`.

```
Promise.resolve(42).then(value => console.log(value));
```

6. Promise.reject(reason)

Returns a promise already rejected with `reason`.

```
Promise.reject("Error occurred").catch(err => console.log(err));
```

7. Promise.all(iterable)

Waits for all promises to resolve. Rejects immediately if any fails.

```
const p1 = Promise.resolve(10);
const p2 = Promise.resolve(20);
Promise.all([p1, p2]).then(values => console.log(values));
```

8. Promise.allSettled(iterable)

Waits for all promises to settle. Returns array of results.

```
const p3 = Promise.resolve(10);
const p4 = Promise.reject("Failed");
Promise.allSettled([p3, p4]).then(results => console.log(results));
```

9. Promise.race(iterable)

Resolves/rejects as soon as one promise settles.

```
const p5 = new Promise(res => setTimeout(() => res("A"), 1000));
const p6 = new Promise(res => setTimeout(() => res("B"), 500));
Promise.race([p5, p6]).then(value => console.log(value));
```

10. Promise.any(iterable)

Resolves as soon as any promise fulfills. Rejects with `AggregateError` if all fail.

```
const p7 = Promise.reject("X");
const p8 = Promise.resolve("Y");
Promise.any([p7, p8]).then(value => console.log(value));
```

11. Quick Summary Table

Method	Description
<code>new Promise()</code>	Create a promise
<code>.then()</code>	Handle fulfilled state
<code>.catch()</code>	Handle rejected state
<code>.finally()</code>	Run code after settled
<code>Promise.resolve()</code>	Returns a resolved promise
<code>Promise.reject()</code>	Returns a rejected promise
<code>Promise.all()</code>	Wait for all promises to fulfill
<code>Promise.allSettled()</code>	Wait for all promises to settle, get result of each
<code>Promise.race()</code>	Returns first settled promise

Method	Description
<code>Promise.any()</code>	Returns first fulfilled promise, ignores rejections